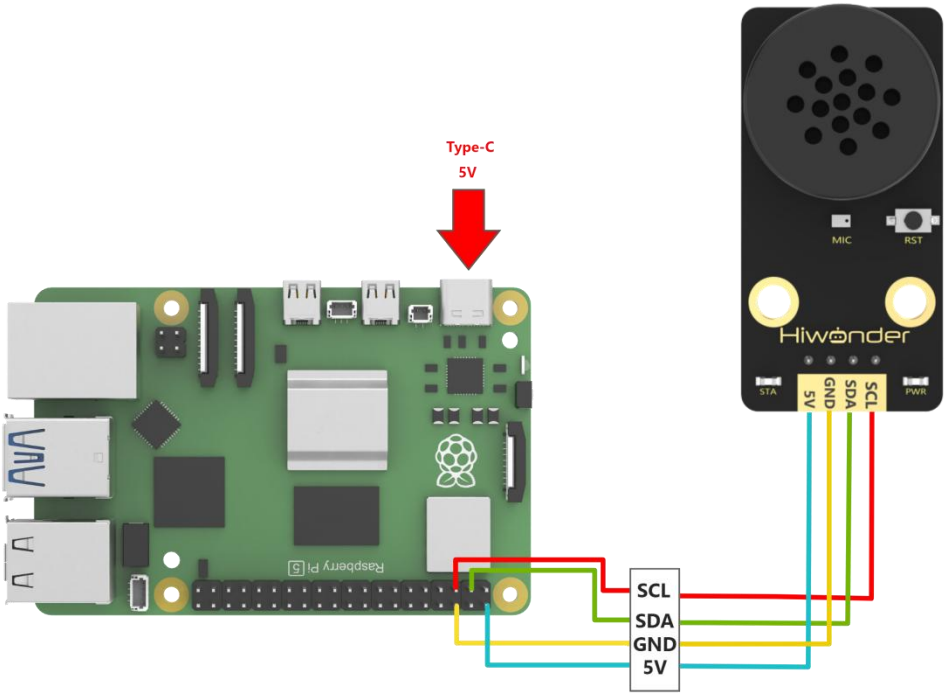
01 Raspberry Pi Voice Broadcast

1. Preparation

1.1 Wiring Setup

When wiring, the 5V, GND, SDA, and SCL pins of the voice interaction module need to be connected to the Raspberry Pi. The wiring method is shown in the image below:



Note:

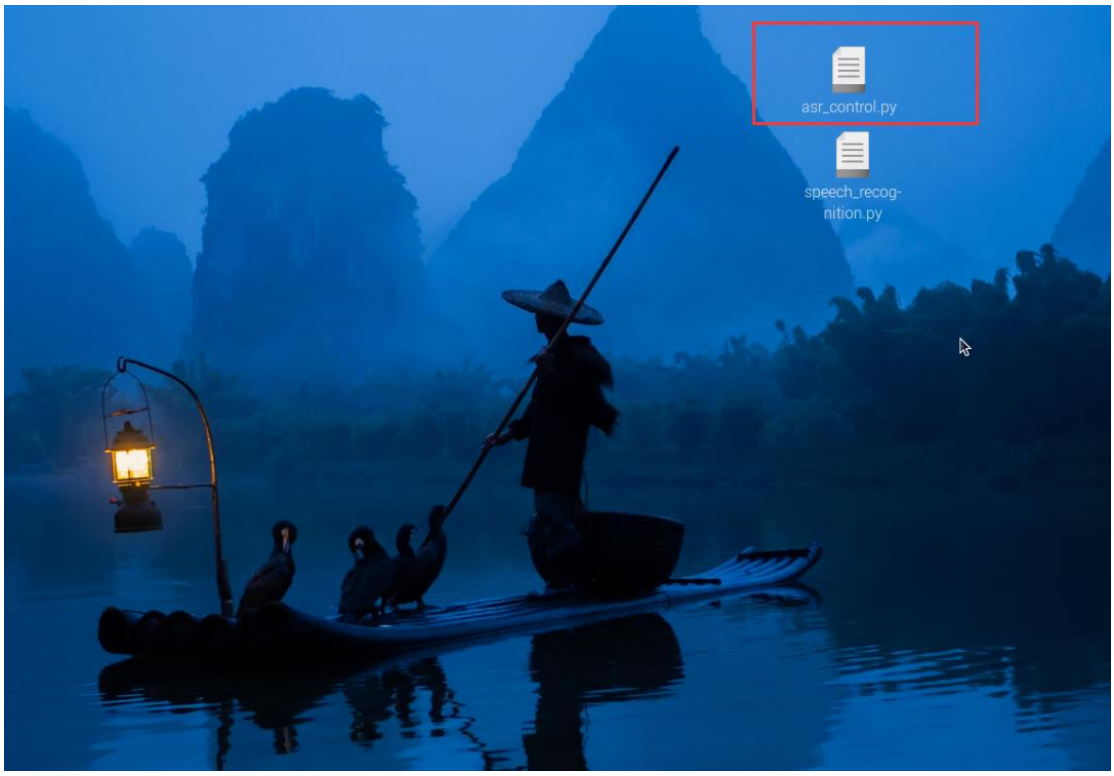
1. **Before** powering on, make sure no metal objects come into contact with the motherboard, as this could cause a short circuit from the pins on the underside, potentially damaging the board.

2. The I2C pin configuration on the Raspberry Pi 4 is identical to that on the **Raspberry** Pi 5, so this wiring method is also compatible with the Raspberry Pi 4.

1.2 Importing and Running the Program

Note: For instructions on installing and using MobaXterm and VNC Viewer, refer to "4.6.1 Remote Connectivity and File Transfer\01 Remote Connectivity and File Transfer."

1) Use MobaXterm to transfer the "speech_recognition.py" file from the "02 Program Files" folder (located in the same directory as this document) to the Raspberry Pi. In this example, we drag the file to the desktop. Then, connect to the Raspberry Pi using VNC Viewer.



2) On the Raspberry Pi, press **Ctrl + Alt + T** to open the terminal.

3) Navigate to the directory where the program is stored by entering:

cd Desktop/

4) Run the program with the following command:

python3 asr_control.py

```
python3 asr_control.py
```

2. Test Case

This routine enables the Raspberry Pi development board to control the voice interaction module, making it broadcast voice messages in a loop. These messages include both command keywords and general broadcast phrases.

2.1 Implementation Overview

Every 5 seconds, the module sequentially announces:

- Moving forward
- Turning left
- Recyclable waste
- Hazardous waste

2.2 Brief Program Analysis

1) Library Imports: The program imports the **smbus** library for I2C bus communication and the **time** library for time processing. These libraries facilitate communication with the voice interaction module.

```
#!/usr/bin/python3
# coding=utf8
import smbus
import time
```

2) Object Creation: An object named asr_module is instantiated from the ASRModule class. During initialization, the I2C address of the integrated

voice interaction module is provided to establish a communication link with the Raspberry Pi bus.

```
if __name__ == "__main__":
    asr_module = ASRModule(I2C_ADDR)
```

3) Class Initialization: When the ASRModule class is instantiated, it automatically creates a member object bus by instantiating the SMBus class from the smbus library. This enables communication with the Raspberry Pi's I2C bus 1. The provided I2C slave address is specified for communication during initialization.

```
class ASRModule:
    def __init__(self,address, bus=1):
        # Initialize I2C bus and device address
        self.bus = smbus.SMBus(bus)  # Use I2C bus 1
        self.address = address  # Device's I2C address
        self.send = [0, 0]  # Initialize the data array for sending
```

4) Writing Data to Registers: This section focuses on writing keyword data to the module registers to initiate voice broadcasts. To achieve this, a function named wire_write_data_array is defined within the ASRModule class to send data to the slave device via the I2C bus.

- The function accepts three parameters: the register address **reg**, a list **val** containing the data to be written, and the data length **length**.
- It leverages the **write_i2c_block_data()** function from the **bus** (an **SMBus** object) to write data to the specified register.

```python
def wire_write_byte(self, val):
    """
    Write a single byte to the device
    :param val: The byte value to be written
    :return: Returns True if the write is successful, False if it fails
    """
    try:
        self.bus.write_byte(self.address, val) # Send byte to the device
        return True # Write successful
    except IOError:
        return False # Write failed, return False
```

5) Using the speak() Function: The speak() function in the ASRModule class handles the writing of keyword data.

- The function first validates the input parameters, specifically whether the keyword type **cmd** matches the two predefined types.
- Once validated, the **cmd** (keyword type) and **id** (keyword ID) are sequentially written into the **send** list of the **ASRModule** class.
- The **wire_write_data_array()** function is then called with the **send** list as its parameter, completing the data writing process.

```python
def speak(self, cmd, id):
    """
    Send a speech command to the device
    :param cmd: The command byte
    :param id: The speech ID
    """
    if cmd == ASR_ANNOUNCER or cmd == ASR_CMDMAND: # Check if the command is valid
        self.send[0] = cmd # Set the first element of the send array to the command
        self.send[1] = id # Set the second element of the send array to the ID
        self.wire_write_data_array(ASR_SPEAK_ADDR, self.send, 2) # Send the command and ID to the specified register
```

6) Main Function Execution: After instantiating the ASRModule class in the main function, a list is defined to store the keyword data to be broadcast. Each element in the list is a tuple containing the keyword type and ID.

- An **ASR_ANNOUNCER** value of **0xFF** indicates general broadcast phrases. Writing **0x01** to the **ANNOUNCER** register corresponds to the broadcast phrase "Recyclable waste," while **0x03** corresponds to "Hazardous waste."

- An **ASR_ANNOUNCER** value of **0x00** represents command keyword broadcasts. Writing **0x01** corresponds to "Moving forward," while **0x02** corresponds to "Turning left."

```
# Define the broadcast content and its corresponding ID
announcements = [
    (ASR_CMDMAND, 1),  # Moving forward
    (ASR_CMDMAND, 3),  # Turning left
    (ASR_ANNOUNCER, 1),  # Recyclable material
    (ASR_ANNOUNCER, 3)   # Hazardous waste
]
```

7) Continuous Loop Execution: The main function then enters a while loop that continuously runs the internal code. Inside this loop, a for loop iterates over each tuple in the announcements list.

- Each tuple is unpacked into the variables **cmd** and **id**.
- The **speak()** function of the **asr_module** object is called with **cmd** and **id** as parameters to broadcast the corresponding keyword.
- After each broadcast, the program sleeps for 5 seconds before proceeding to the next tuple.
- Once the list is fully traversed, the **while** loop restarts, repeating the broadcast sequence indefinitely.

```
while True:
    for cmd, id in announcements:
        asr_module.speak(cmd, id)
        time.sleep(5)
```

For a detailed list of keyword data, refer to the **"4.3 Command Word Broadcasting Protocol List".**